



RAPPORT DE PROJET

Développement d'un système intelligent de vérification automatique de faits basé sur l'IA (Fact-Checker AI)

Nom : ASSAN CYRIAC ABONOUAN

Établissement : TECH TALENT ACCELERATOR

Formation : GenAI & Machine Learning

Formateur : Mr. ELINGUI PASCAL URIEL

Date : 29/07/2026

Résumé

Ce projet vise à concevoir et développer un système de vérification automatique de faits (Fact-Checker AI), capable d'évaluer la véracité d'une affirmation en s'appuyant sur un corpus documentaire Wikipedia indexé. Le système repose sur une architecture de type recherche augmentée (RAG) associant recherche sémantique par embeddings (Sentence Transformers, modèle all-MiniLM-L6-v2), un index vectoriel FAISS, un module de reranking par cross-encoder (ms-marco-MiniLM-L-6-v2) et un module de vérification par inférence de langage naturel (NLI, modèle nli-deberta-v3-base).

Le pipeline complet, orchestré par la classe FactCheckerPipeline, transforme une affirmation en un verdict (SUPPORTS, REFUTES ou NOT ENOUGH INFO) accompagné des preuves retrouvées et d'un score de confiance, restitué à l'utilisateur via une interface Streamlit.

Le système a été évalué sur le jeu de développement FEVER avec la commande python `evaluation/evaluate_pipeline.py`. Sur 19 998 exemples chargés, 24 affirmations étaient évaluables avec le corpus indexé courant et 19 974 ont été ignorées faute de preuve de référence exploitable dans l'index. Le pipeline atteint une exactitude (accuracy) de 75,00 %, un taux de rappel des preuves (Evidence Recall) de 54,17 %, une Evidence Precision de 3,34 %, un FEVER Score de 41,67 %, un Macro F1 de 0,5211 et un ECE de 0,1333. Ce rapport présente le contexte du projet, la problématique traitée, l'architecture et le fonctionnement détaillé du système, son interface utilisateur, les résultats d'évaluation obtenus, ainsi que les difficultés rencontrées, les limites actuelles et les perspectives d'évolution.

1. Contexte

- L'information circule aujourd'hui à une vitesse et dans un volume sans précédent, notamment via Internet et les réseaux sociaux.
- Cette expansion rapide s'accompagne d'une propagation tout aussi rapide de fausses informations (« fake news »), parfois difficiles à distinguer de sources fiables.
- Face à ce flux constant, les utilisateurs disposent rarement du temps ou des outils nécessaires pour vérifier rapidement et de manière fiable la véracité d'une affirmation.
- Ce contexte a conduit au développement de systèmes automatisés de fact-checking, capables de croiser une affirmation avec une base documentaire de référence afin d'en évaluer la véracité de manière rapide, reproductible et transparente.

2. Problématique

Comment développer un système capable de vérifier automatiquement la véracité d'une affirmation en s'appuyant sur une base documentaire fiable, tout en fournissant des preuves et une explication compréhensible?

3. Solution proposée

Fact-Checker AI est un système de vérification automatique de faits qui s'appuie sur un corpus documentaire Wikipedia indexé, et sur le jeu de données FEVER (Fact Extraction and VERification) pour son évaluation.

Principe du système

À partir d'une affirmation (claim) soumise par l'utilisateur, le système recherche dans le corpus indexé les passages les plus pertinents, les trie par pertinence, puis analyse chaque passage retenu à l'aide d'un modèle d'inférence de langage naturel (NLI) afin de déterminer s'il confirme, contredit, ou ne permet pas de statuer sur l'affirmation. Les résultats individuels sont ensuite agrégés pour produire un verdict global assorti d'un score de confiance.

Fonctionnement global

Affirmation → Recherche (retrieval) → Tri des preuves (reranking) → Vérification (NLI) → Verdict et preuves.

Utilisateurs ciblés

Toute personne souhaitant vérifier rapidement une affirmation à l'aide de sources documentaires (contexte pédagogique, veille informationnelle, aide à la vérification). Le système est explicitement présenté, dans son interface, comme une aide à la vérification et non comme une source de vérité unique.

Bénéfices

- Transparence : chaque verdict est accompagné des preuves et du document source utilisés.

- Traçabilité : chaque étape du pipeline (recherche, tri, analyse) est indépendante et peut être vérifiée séparément.
- Accessibilité : interface web simple ne nécessitant aucune compétence technique.

4. Objectifs

Objectif général

Concevoir et développer un pipeline complet de vérification automatique de faits, combinant recherche d'information sémantique et vérification par intelligence artificielle, restitué via une interface utilisateur accessible.

Objectifs spécifiques

- Prétraiter le corpus Wikipedia (segmentation en phrases et en chunks) — `indexing/preprocess_wikipedia.py`
- Construire une base documentaire structurée — `indexing/build_chunk_database.py` (base SQLite `chunks.db`)
- Générer les représentations vectorielles (embeddings) des passages — `indexing/build_embeddings.py`
- Construire un index de recherche vectorielle FAISS — `indexing/build_faiss.py`
- Retrouver les documents et passages pertinents pour une affirmation donnée — `retrieval/document_retriever.py`, `retrieval/passage_retriever.py`
- Trier les preuves retrouvées par pertinence (reranking) — `retrieval/reranker.py`
- Vérifier une affirmation grâce à l'IA (inférence de langage naturel) — `verification/verifier.py`
- Générer une explication du verdict à travers les preuves et les scores de confiance associés — `frontend/app.py`

5. Technologies utilisées

Technologie	Utilisation
Python	Langage de développement principal du projet
Streamlit	Interface utilisateur web
FAISS (faiss-cpu)	Recherche vectorielle (index de similarité)
Sentence Transformers	Génération des embeddings (modèle <code>all-MiniLM-L6-v2</code>)
Cross-Encoder	Reranking des preuves (modèle <code>ms-marco-MiniLM-L-6-v2</code>)
Transformers (Hugging Face)	Vérification NLI (modèle <code>nli-deberta-v3-base</code>)
PyTorch	Backend d'exécution des modèles de deep learning
NumPy	Manipulation et stockage des embeddings
SQLite	Base de données des passages indexés (<code>chunks.db</code>)
FEVER	Jeu de données d'évaluation (affirmations, preuves, labels)

6. Architecture générale

```

1
2 @dataclass(frozen=True)
3 class PipelineSettings:
4     retrieval_top_k: int = DEFAULT_RETRIEVAL_TOP_K
5     rerank_top_k: int = DEFAULT_RERANK_TOP_K
6     max_chunks_per_document: int = DEFAULT_MAX_CHUNKS_PER_DOCUMENT
7
8
9 class FactCheckerPipeline:
10     def __init__(
11         self,
12         document_retriever: DocumentRetriever | None = None,
13         passage_retriever: PassageRetriever | None = None,
14         reranker: Reranker | None = None,
15         verifier: Verifier | None = None,
16         settings: PipelineSettings | None = None,
17     ):
18         self.settings = settings or PipelineSettings()
19         self.document_retriever = document_retriever or DocumentRetriever()
20         self.passage_retriever = passage_retriever or PassageRetriever(
21             self.document_retriever
22         )
23         self.reranker = reranker or Reranker()
24         self.verifier = verifier or Verifier()
25
26     def verify_claim(self, claim: str) -> dict:
27         claim = claim.strip()
28         if not claim:
29             raise ValueError("Claim must not be empty.")
30
31         passages = self.passage_retriever.retrieve(
32             query=claim,
33             top_k=self.settings.retrieval_top_k,
34             max_chunks_per_document=self.settings.max_chunks_per_document,
35         )
36         reranked_passages = self.reranker.rerank(
37             query=claim,
38             passages=passages,
39             top_k=self.settings.rerank_top_k,
40         )
41         verification = self.verifier.verify(
42             claim=claim,
43             evidence=reranked_passages,
44         )
45
46         return {
47             "claim": claim,
48             "label": verification["label"],
49             "confidence": verification["confidence"],
50             "label_scores": verification["label_scores"],
51             "best_evidence": verification["best_evidence"],
52             "evidence": verification["evidence"],
53             "retrieved_passages": passages,
54             "reranked_passages": reranked_passages,
55             "settings": {
56                 "retrieval_top_k": self.settings.retrieval_top_k,
57                 "rerank_top_k": self.settings.rerank_top_k,
58                 "max_chunks_per_document": self.settings.max_chunks_per_document,
59             },
60         }
61
62     def close(self) -> None:
63         self.document_retriever.close()
64

```

Le pipeline de vérification est orchestré par la classe FactCheckerPipeline (api/pipeline.py), qui enchaîne quatre composants indépendants: DocumentRetriever (recherche vectorielle), PassageRetriever (mise en forme et diversification des passages), Reranker (tri fin par cross-

encoder) et Verifier (vérification NLI). Ce découpage en étapes indépendantes permet de tester et d'évaluer chaque composant séparément.

Le pipeline suit le schéma suivant :

Affirmation → Recherche → Tri → Preuves → Analyse → Verdict

7. Fonctionnement détaillé

7.1 Prétraitement

```
1
2 def load_articles(wiki_dir: Path) -> Iterator[dict]:
3     """
4     Parcourt tous les fichiers wiki-*.jsonl et retourne les articles
5     un par un afin d'éviter de charger tout le corpus en memoire.
6     """
7
8     files = sorted(wiki_dir.glob("wiki-*.jsonl"))
9     if MAX_FILES is not None:
10        files = files[:MAX_FILES]
11
12    for file_path in files:
13
14        with file_path.open(
15            "r",
16            encoding="utf-8"
17        ) as f:
18            for line in f:
19                if line.strip():
20                    yield json.loads(line)
21
22
23 def extract_sentences(article: dict) -> List[dict]:
24     """
25     Extrait les phrases d'un article Wikipedia.
26
27     Parameters
28     -----
29     article : dict
30         Article provenant du dataset Wikipedia.
31
32     Returns
33     -----
34     List[dict]
35         Liste de phrases au format {"id": sentence_id, "text": sentence_text}.
36     """
37
38    sentences = []
39
40    lines = article.get("lines", "")
41
42    if not lines:
43        return sentences
44
45    for line in lines.split("\n"):
46
47        line = line.strip()
48
49        if not line:
50            continue
51
52        parts = line.split("\t", maxsplit=1)
53
54        if len(parts) != 2:
55            continue
56
57        sentence_id, sentence = parts
58
59        try:
60            sentence = sentence.replace("_", " ")
61
62
63
64
65
66
67
68
69
70
71
72
73
74
75
76
77
78
79
80
81
82
83
84
85
86
87
88
89
90
91
92
93
94
95
96
97
98
99
100
101
102
103
104
105
106
107
108
109
110
111
112
113
114
115
116
117
118
119
120
121
122
123
124
125
126
127
128
129
130
131
132
133
134
135
136
137
138
139
140
141
142
143
144
145
146
147
148
149
150
151
152
153
154
155
156
157
158
159
160
161
162
163
164
165
166
167
168
169
170
171
172
173
174
175
176
177
178
179
180
181
182
183
184
185
186
187
188
189
190
191
192
193
194
195
196
197
198
199
200
201
202
203
204
205
206
207
208
209
210
211
212
213
214
215
216
217
218
219
220
221
222
223
224
225
226
227
228
229
230
231
232
233
234
235
236
237
238
239
240
241
242
243
244
245
246
247
248
249
250
251
252
253
254
255
256
257
258
259
260
261
262
263
264
265
266
267
268
269
270
271
272
273
274
275
276
277
278
279
280
281
282
283
284
285
286
287
288
289
290
291
292
293
294
295
296
297
298
299
300
301
302
303
304
305
306
307
308
309
310
311
312
313
314
315
316
317
318
319
320
321
322
323
324
325
326
327
328
329
330
331
332
333
334
335
336
337
338
339
340
341
342
343
344
345
346
347
348
349
350
351
352
353
354
355
356
357
358
359
360
361
362
363
364
365
366
367
368
369
370
371
372
373
374
375
376
377
378
379
380
381
382
383
384
385
386
387
388
389
390
391
392
393
394
395
396
397
398
399
400
401
402
403
404
405
406
407
408
409
410
411
412
413
414
415
416
417
418
419
420
421
422
423
424
425
426
427
428
429
430
431
432
433
434
435
436
437
438
439
440
441
442
443
444
445
446
447
448
449
450
451
452
453
454
455
456
457
458
459
460
461
462
463
464
465
466
467
468
469
470
471
472
473
474
475
476
477
478
479
480
481
482
483
484
485
486
487
488
489
490
491
492
493
494
495
496
497
498
499
500
501
502
503
504
505
506
507
508
509
510
511
512
513
514
515
516
517
518
519
520
521
522
523
524
525
526
527
528
529
530
531
532
533
534
535
536
537
538
539
540
541
542
543
544
545
546
547
548
549
550
551
552
553
554
555
556
557
558
559
560
561
562
563
564
565
566
567
568
569
570
571
572
573
574
575
576
577
578
579
580
581
582
583
584
585
586
587
588
589
590
591
592
593
594
595
596
597
598
599
600
601
602
603
604
605
606
607
608
609
610
611
612
613
614
615
616
617
618
619
620
621
622
623
624
625
626
627
628
629
630
631
632
633
634
635
636
637
638
639
640
641
642
643
644
645
646
647
648
649
650
651
652
653
654
655
656
657
658
659
660
661
662
663
664
665
666
667
668
669
670
671
672
673
674
675
676
677
678
679
680
681
682
683
684
685
686
687
688
689
690
691
692
693
694
695
696
697
698
699
700
701
702
703
704
705
706
707
708
709
710
711
712
713
714
715
716
717
718
719
720
721
722
723
724
725
726
727
728
729
730
731
732
733
734
735
736
737
738
739
740
741
742
743
744
745
746
747
748
749
750
751
752
753
754
755
756
757
758
759
760
761
762
763
764
765
766
767
768
769
770
771
772
773
774
775
776
777
778
779
780
781
782
783
784
785
786
787
788
789
790
791
792
793
794
795
796
797
798
799
800
801
802
803
804
805
806
807
808
809
810
811
812
813
814
815
816
817
818
819
820
821
822
823
824
825
826
827
828
829
830
831
832
833
834
835
836
837
838
839
840
841
842
843
844
845
846
847
848
849
850
851
852
853
854
855
856
857
858
859
860
861
862
863
864
865
866
867
868
869
870
871
872
873
874
875
876
877
878
879
880
881
882
883
884
885
886
887
888
889
890
891
892
893
894
895
896
897
898
899
900
901
902
903
904
905
906
907
908
909
910
911
912
913
914
915
916
917
918
919
920
921
922
923
924
925
926
927
928
929
930
931
932
933
934
935
936
937
938
939
940
941
942
943
944
945
946
947
948
949
950
951
952
953
954
955
956
957
958
959
960
961
962
963
964
965
966
967
968
969
970
971
972
973
974
975
976
977
978
979
980
981
982
983
984
985
986
987
988
989
990
991
992
993
994
995
996
997
998
999
1000
1001
1002
1003
1004
1005
1006
1007
1008
1009
1010
1011
1012
1013
1014
1015
1016
1017
1018
1019
1020
1021
1022
1023
1024
1025
1026
1027
1028
1029
1030
1031
1032
1033
1034
1035
1036
1037
1038
1039
1040
1041
1042
1043
1044
1045
1046
1047
1048
1049
1050
1051
1052
1053
1054
1055
1056
1057
1058
1059
1060
1061
1062
1063
1064
1065
1066
1067
1068
1069
1070
1071
1072
1073
1074
1075
1076
1077
1078
1079
1080
1081
1082
1083
1084
1085
1086
1087
1088
1089
1090
1091
1092
1093
1094
1095
1096
1097
1098
1099
1100
1101
1102
1103
1104
1105
1106
1107
1108
1109
1110
1111
1112
1113
1114
1115
1116
1117
1118
1119
1120
1121
1122
1123
1124
1125
1126
1127
1128
1129
1130
1131
1132
1133
1134
1135
1136
1137
1138
1139
1140
1141
1142
1143
1144
1145
1146
1147
1148
1149
1150
1151
1152
1153
1154
1155
1156
1157
1158
1159
1160
1161
1162
1163
1164
1165
1166
1167
1168
1169
1170
1171
1172
1173
1174
1175
1176
1177
1178
1179
1180
1181
1182
1183
1184
1185
1186
1187
1188
1189
1190
1191
1192
1193
1194
1195
1196
1197
1198
1199
1200
1201
1202
1203
1204
1205
1206
1207
1208
1209
1210
1211
1212
1213
1214
1215
1216
1217
1218
1219
1220
1221
1222
1223
1224
1225
1226
1227
1228
1229
1230
1231
1232
1233
1234
1235
1236
1237
1238
1239
1240
1241
1242
1243
1244
1245
1246
1247
1248
1249
1250
1251
1252
1253
1254
1255
1256
1257
1258
1259
1260
1261
1262
1263
1264
1265
1266
1267
1268
1269
1270
1271
1272
1273
1274
1275
1276
1277
1278
1279
1280
1281
1282
1283
1284
1285
1286
1287
1288
1289
1290
1291
1292
1293
1294
1295
1296
1297
1298
1299
1300
1301
1302
1303
1304
1305
1306
1307
1308
1309
1310
1311
1312
1313
1314
1315
1316
1317
1318
1319
1320
1321
1322
1323
1324
1325
1326
1327
1328
1329
1330
1331
1332
1333
1334
1335
1336
1337
1338
1339
1340
1341
1342
1343
1344
1345
1346
1347
1348
1349
1350
1351
1352
1353
1354
1355
1356
1357
1358
1359
1360
1361
1362
1363
1364
1365
1366
1367
1368
1369
1370
1371
1372
1373
1374
1375
1376
1377
1378
1379
1380
1381
1382
1383
1384
1385
1386
1387
1388
1389
1390
1391
1392
1393
1394
1395
1396
1397
1398
1399
1400
1401
1402
1403
1404
1405
1406
1407
1408
1409
1410
1411
1412
1413
1414
1415
1416
1417
1418
1419
1420
1421
1422
1423
1424
1425
1426
1427
1428
1429
1430
1431
1432
1433
1434
1435
1436
1437
1438
1439
1440
1441
1442
1443
1444
1445
1446
1447
1448
1449
1450
1451
1452
1453
1454
1455
1456
1457
1458
1459
1460
1461
1462
1463
1464
1465
1466
1467
1468
1469
1470
1471
1472
1473
1474
1475
1476
1477
1478
1479
1480
1481
1482
1483
1484
1485
1486
1487
1488
1489
1490
1491
1492
1493
1494
1495
1496
1497
1498
1499
1500
1501
1502
1503
1504
1505
1506
1507
1508
1509
1510
1511
1512
1513
1514
1515
1516
1517
1518
1519
1520
1521
1522
1523
1524
1525
1526
1527
1528
1529
1530
1531
1532
1533
1534
1535
1536
1537
1538
1539
1540
1541
1542
1543
1544
1545
1546
1547
1548
1549
1550
1551
1552
1553
1554
1555
1556
1557
1558
1559
1560
1561
1562
1563
1564
1565
1566
1567
1568
1569
1570
1571
1572
1573
1574
1575
1576
1577
1578
1579
1580
1581
1582
1583
1584
1585
1586
1587
1588
1589
1590
1591
1592
1593
1594
1595
1596
1597
1598
1599
1600
1601
1602
1603
1604
1605
1606
1607
1608
1609
1610
1611
1612
1613
1614
1615
1616
1617
1618
1619
1620
1621
1622
1623
1624
1625
1626
1627
1628
1629
1630
1631
1632
1633
1634
1635
1636
1637
1638
1639
1640
1641
1642
1643
1644
1645
1646
1647
1648
1649
1650
1651
1652
1653
1654
1655
1656
1657
1658
1659
1660
1661
1662
1663
1664
1665
1666
1667
1668
1669
1670
1671
1672
1673
1674
1675
1676
1677
1678
1679
1680
1681
1682
1683
1684
1685
1686
1687
1688
1689
1690
1691
1692
1693
1694
1695
1696
1697
1698
1699
1700
1701
1702
1703
1704
1705
1706
1707
1708
1709
1710
1711
1712
1713
1714
1715
1716
1717
1718
1719
1720
1721
1722
1723
1724
1725
1726
1727
1728
1729
1730
1731
1732
1733
1734
1735
1736
1737
1738
1739
1740
1741
1742
1743
1744
1745
1746
1747
1748
1749
1750
1751
1752
1753
1754
1755
1756
1757
1758
1759
1760
1761
1762
1763
1764
1765
1766
1767
1768
1769
1770
1771
1772
1773
1774
1775
1776
1777
1778
1779
1780
1781
1782
1783
1784
1785
1786
1787
1788
1789
1790
1791
1792
1793
1794
1795
1796
1797
1798
1799
1800
1801
1802
1803
1804
1805
1806
1807
1808
1809
1810
1811
1812
1813
1814
1815
1816
1817
1818
1819
1820
1821
1822
1823
1824
1825
1826
1827
1828
1829
1830
1831
1832
1833
1834
1835
1836
1837
1838
1839
1840
1841
1842
1843
1844
1845
1846
1847
1848
1849
1850
1851
1852
1853
1854
1855
1856
1857
1858
1859
1860
1861
1862
1863
1864
1865
1866
1867
1868
1869
1870
1871
1872
1873
1874
1875
1876
1877
1878
1879
1880
1881
1882
1883
1884
1885
1886
1887
1888
1889
1890
1891
1892
1893
1894
1895
1896
1897
1898
1899
1900
1901
1902
1903
1904
1905
1906
1907
1908
1909
1910
1911
1912
1913
1914
1915
1916
1917
1918
1919
1920
1921
1922
1923
1924
1925
1926
1927
1928
1929
1930
1931
1932
1933
1934
1935
1936
1937
1938
1939
1940
1941
1942
1943
1944
1945
1946
1947
1948
1949
1950
1951
1952
1953
1954
1955
1956
1957
1958
1959
1960
1961
1962
1963
1964
1965
1966
1967
1968
1969
1970
1971
1972
1973
1974
1975
1976
1977
1978
1979
1980
1981
1982
1983
1984
1985
1986
1987
1988
1989
1990
1991
1992
1993
1994
1995
1996
1997
1998
1999
2000
2001
2002
2003
2004
2005
2006
2007
2008
2009
2010
2011
2012
2013
2014
2015
2016
2017
2018
2019
2020
2021
2022
2023
2024
2025
2026
2027
2028
2029
2030
2031
2032
2033
2034
2035
2036
2037
2038
2039
2040
2041
2042
2043
2044
2045
2046
2047
2048
2049
2050
2051
2052
2053
2054
2055
2056
2057
2058
2059
2060
2061
2062
2063
2064
2065
2066
2067
2068
2069
2070
2071
2072
2073
2074
2075
2076
2077
2078
2079
2080
2081
2082
2083
2084
2085
2086
2087
2088
2089
2090
2091
2092
2093
2094
2095
2096
2097
2098
2099
2100
2101
2102
2103
2104
2105
2106
2107
2108
2109
2110
2111
2112
2113
2114
2115
2116
2117
2118
2119
2120
2121
2122
2123
2124
2125
2126
2127
2128
2129
2130
2131
2132
2133
2134
2135
2136
2137
2138
2139
2140
2141
2142
2143
2144
2145
2146
2147
2148
2149
2150
2151
2152
2153
2154
2155
2156
2157
2158
2159
2160
2161
2162
2163
2164
2165
2166
2167
2168
2169
2170
2171
2172
2173
2174
2175
2176
2177
2178
2179
2180
2181
2182
2183
2184
2185
2186
2187
2188
2189
2190
2191
2192
2193
2194
2195
2196
2197
2198
2199
2200
2201
2202
2203
2204
2205
2206
2207
2208
2209
2210
2211
2212
2213
2214
2215
2216
2217
2218
2219
2220
2221
2222
2223
2224
2225
2226
2227
2228
2229
2230
2231
2232
2233
2234
2235
2236
2237
2238
2239
2240
2241
2242
2243
2244
2245
2246
2247
2248
2249
2250
2251
2252
2253
2254
2255
2256
2257
2258
2259
2260
2261
2262
2263
2264
2265
2266
2267
2268
2269
2270
2271
2272
2273
2274
2275
2276
2277
2278
2279
2280
2281
2282
2283
2284
2285
2286
2287
2288
2289
2290
2291
2292
2293
2294
2295
2296
2297
2298
2299
2300
2301
2302
2303
2304
2305
2306
2307
2308
2309
2310
2311
2312
2313
2314
2315
2316
2317
2318
2319
2320
2321
2322
2323
2324
2325
2326
2327
2328
2329
2330
2331
2332
2333
2334
2335
2336
2337
2338
2339
2340
2341
2342
2343
2344
2345
2346
2347
2348
2349
2350
2351
2352
2353
2354
2355
2356
2357
2358
2359
2360
2361
2362
2363
2364
2365
2366
2367
2368
2369
2370
2371
2372
2373
2374
2375
2376
2377
2378
2379
2380
2381
2382
2383
2384
2385
2386
2387
2388
2389
2390
2391
2392
2393
2394
2395
2396
2397
2398
2399
2400
2401
2402
2403
2404
2405
2406
2407
2408
2409
2410
2411
2412
2413
2414
2415
2416
2417
2418
2419
2420
2421
2422
2423
2424
2425
2426
2427
2428
2429
2430
2431
2432
2433
2434
2435
2436
2437
2438
2439
2440
2441
2442
2443
2444
2445
2446
2447
2448
2449
2450
2451
2452
2453
2454
2455
2456
2457
2458
2459
2460
2461
2462
2463
2464
2465
2466
2467
2468
2469
2470
2471
2472
2473
2474
2475
2476
2477
2478
2479
2480
2481
2482
2483
2484
2485
2486
2487
2488
2489
2490
2491
2492
2493
2494
2495
2496
2497
2498
2499
2500
2501
2502
2503
2504
2505
2506
2507
2508
2509
2510
2511
2512
2513
2514
2515
2516
2517
2518
2519
2520
2521
2522
2523
2524
2525
2526
2527
2528
2529
2530
2531
2532
2533
2534
2535
2536
2537
2538
2539
2540
2541
2542
2543
2544
2545
2546
2547
2548
2549
2550
2551
2552
2553
2554
2555
2556
2557
2558
2559
2560
2561
2562
2563
2564
2565
2566
2567
2568
2569
2570
2571
2572
2573
2574
2575
2576
2577
2578
2579
2580
2581
2582
2583
2584
2585
2586
25
```

7.2 Génération des embeddings

```
1
2 def load_chunks(input_file: Path) -> Iterator[dict]:
3     """
4     Lit les chunks un par un depuis le fichier JSONL.
5     """
6
7     with input_file.open(
8         "r",
9         encoding="utf-8"
10    ) as f:
11        for line in f:
12            if line.strip():
13                yield json.loads(line)
14
15 def load_model(model_name: str) -> SentenceTransformer:
16     """
17     Charge le modèle SentenceTransformer utilisé
18     pour générer les embeddings.
19     """
20     print("Chargement du modèle...")
21
22     model = SentenceTransformer(model_name)
23
24     print("Modèle chargé.")
25
26     return model
27
28
29 def encode_batch(
30     model: SentenceTransformer,
31     texts: list[str]
32 ) -> np.ndarray:
33     """
34     Encode un batch de textes en embeddings.
35     """
36
37     embeddings = model.encode(
38         texts,
39         batch_size=len(texts),
40         show_progress_bar=False, # la pipeline aura déjà sa propre progression dans main().
41         convert_to_numpy=True, # on évite donc une conversion supplémentaire,
42         normalize_embeddings=True # En normalisant des cette étape : on évite de le refaire plus tard ; la recherche est plus rapide ; les scores sont plus cohérents. C'est une pratique courante avec les modèles Sentence Transformers.
43     )
44
45     return embeddings
46
47
48
```

Le script `indexing/build_embeddings.py` charge le modèle `SentenceTransformer all-MiniLM-L6-v2` et encode les chunks par lots de 512 (`BATCH_SIZE = 512`). Les embeddings sont normalisés au moment de l'encodage, ce qui permet ensuite une recherche par similarité cosinus efficace. Les résultats sont sauvegardés sous forme de plusieurs fichiers `.numpy`, accompagnés d'un fichier de métadonnées (`metadata.json`) décrivant le modèle utilisé et la dimension des vecteurs.

7.3 Construction de l'index FAISS

```
1
2 def load_metadata(metadata_file: Path) -> dict:
3     """
4     Charge les métadonnées des embeddings.
5     """
6
7     with metadata_file.open("r",encoding="utf-8") as f:
8
9         return json.load(f)
10
11 def load_embedding_files(embeddings_dir: Path) -> Iterator[Path]:
12     """
13     Parcourt tous les fichiers d'embeddings (.npz)
14     dans l'ordre de leur création.
15     """
16
17     for file_path in sorted(embeddings_dir.glob("embeddings_*.npz")):
18         yield file_path
19
20
21 def create_index(embedding_dimension: int) -> faiss.Index:
22     """
23     Crée un index FAISS basé sur
24     la similarité cosinus.
25     """
26
27     return faiss.IndexFlatIP(embedding_dimension) #tous les vecteurs ont une norme égale à 1
28
29 def add_embeddings(index: faiss.Index, embedding_file: Path ) -> int:
30     """
31     Charge un fichier d'embeddings
32     et les ajoute à l'index FAISS.
33
34     Returns
35     -----
36     int
37         Nombre de vecteurs ajoutés.
38     """
39
40     embeddings = np.load(
41         embedding_file
42     ).astype(np.float32)
43
44     index.add(
45         embeddings
46     )
47
48     return embeddings.shape[0]
49
50 def save_index(index: faiss.Index,output_file: Path ) -> None:
51     """
52     Sauvegarde l'index FAISS.
53     """
54
55     faiss.write_index(
56         index,
57         str(output_file)
58     )
59
60
```

Construction de l'index FAISS

Le script `indexing/build_faiss.py` charge l'ensemble des fichiers d'embeddings générés, construit un index FAISS de type `IndexFlatIP` (produit scalaire, équivalent à une similarité cosinus puisque les vecteurs sont normalisés), puis sauvegarde l'index dans `data/indexes/faiss.index`.

7.4 Recherche des documents


```

1
2 def _missing_dependency_message(package: str) -> str:
3     return f"Missing dependency '{package}'. Activate the project environment before running retrieval, for example: conda activate fact-checker."
4
5 def load_index(index_file: Path = FAISS_INDEX_FILE) -> Any:
6     if not index_file.exists():
7         raise FileNotFoundError(f"FAISS index not found: {index_file}")
8
9     try:
10         import faiss
11     except ImportError as exc:
12         raise ImportError(_missing_dependency_message("faiss")) from exc
13
14     return faiss.read_index(str(index_file))
15
16 def load_model(model_name: str = EMBEDDING_MODEL_NAME) -> Any:
17     try:
18         from sentence_transformers import SentenceTransformer
19     except ImportError as exc:
20         raise ImportError(_missing_dependency_message("sentence-transformers")) from exc
21
22     return SentenceTransformer(model_name)
23
24 def encode_query(model: Any, query: str) -> np.ndarray:
25     query = query.strip()
26
27     if not query:
28         raise ValueError("Query must not be empty.")
29
30     embedding = model.encode(
31         [query],
32         convert_to_numpy=True,
33         normalize_embeddings=True,
34         show_progress_bar=False,
35     )
36
37     return embedding.astype(np.float32)
38
39 def search_index(index: Any, query_embedding: np.ndarray, top_k: int) -> tuple[np.ndarray, np.ndarray]:
40
41     if top_k <= 0:
42         raise ValueError("top_k must be greater than zero.")
43
44     return index.search(query_embedding, top_k)
45
46 def load_database(database_file: Path = CHUNKS_DATABASE_FILE) -> sqlite3.Connection:
47     if not database_file.exists():
48         raise FileNotFoundError(f"SQLite chunks database not found: {database_file}")
49
50     connection = sqlite3.connect(str(database_file))
51     connection.row_factory = sqlite3.Row
52
53     return connection
54
55 def get_chunk(connection: sqlite3.Connection, chunk_id: int) -> dict | None:
56
57     cursor = connection.execute(
58         """
59         SELECT
60             chunk_id,
61             doc_id,
62             title,
63             source,
64             start_sentence,
65             end_sentence,
66             sentence_ids,
67             num_sentences,
68             text_length,
69             text
70         FROM chunks
71         WHERE chunk_id = ?
72         """,
73         (chunk_id,),
74     )
75
76     row = cursor.fetchone()
77
78     if row is None:
79         return None
80
81     return {
82         "chunk_id": row["chunk_id"],
83         "doc_id": row["doc_id"],
84         "title": row["title"],
85         "source": row["source"],
86         "start_sentence": row["start_sentence"],
87         "end_sentence": row["end_sentence"],
88         "sentence_ids": json.loads(row["sentence_ids"]),
89         "num_sentences": row["num_sentences"],
90         "text_length": row["text_length"],
91         "text": row["text"],
92     }
93
94

```

Recherche de documents

Lorsqu'une affirmation est soumise, DocumentRetriever (retrieval/document_retriever.py) encode la requête avec le même modèle d'embeddings que celui utilisé pour l'indexation, puis interroge l'index FAISS pour récupérer les passages les plus proches sémantiquement. Les chunks correspondants sont ensuite récupérés depuis la base SQLite (chunks.db). PassageRetriever (retrieval/passage_retriever.py) limite ensuite le nombre de passages provenant d'un même document (par défaut 3), afin de diversifier les sources retenues.

7.5 Reranking

```
1 from typing import Any
2
3 from config.setting import RERANKER_MODEL_NAME
4
5 class Reranker:
6     def __init__(self, model_name: str = RERANKER_MODEL_NAME, model: Any | None = None):
7         if model is not None:
8             self.model = model
9             self.model_name = model_name
10            return
11
12        try:
13            from sentence_transformers import CrossEncoder
14        except ImportError as exc:
15            raise ImportError(
16                "Missing dependency 'sentence-transformers'. Activate the project "
17                "environment before running reranking."
18            ) from exc
19
20        self.model_name = model_name
21        self.model = CrossEncoder(model_name)
22
23    def rerank(self, query: str, passages: list[dict], top_k: int = 5) -> list[dict]:
24        query = query.strip()
25        if not query:
26            raise ValueError("Query must not be empty.")
27        if top_k <= 0:
28            raise ValueError("top_k must be greater than zero.")
29        if not passages:
30            return []
31
32        pairs = [(query, passage["text"]) for passage in passages]
33        scores = self.model.predict(pairs)
34
35        reranked_results = []
36        for passage, score in zip(passages, scores):
37            result = passage.copy()
38            result["rerank_score"] = float(score)
39            reranked_results.append(result)
40
41        reranked_results.sort(key=lambda item: item["rerank_score"], reverse=True)
42        return reranked_results[:top_k]
43
```

Reranking des preuves

Le Reranker (retrieval/reranker.py) utilise un modèle Cross-Encoder (ms-marco-MiniLM-L-6-v2) qui évalue directement chaque paire (affirmation, passage), ce qui est plus précis qu'une simple similarité vectorielle. Seuls les meilleurs passages (par défaut 5) sont conservés pour l'étape de vérification.

7.6 Vérification (NLI)

```
1
2 class Verifier:
3     def __init__(
4         self,
5         model_name: str = NLI_MODEL_NAME,
6         tokenizer: Any | None = None,
7         model: Any | None = None,
8     ):
9         self.model_name = model_name
10
11         if tokenizer is not None and model is not None:
12             self.tokenizer = tokenizer
13             self.model = model
14         else:
15             from transformers import AutoModelForSequenceClassification, AutoTokenizer
16
17             self.tokenizer = AutoTokenizer.from_pretrained(model_name)
18             self.model = AutoModelForSequenceClassification.from_pretrained(model_name)
19
20         self.model.eval()
21         self.id2label = dict(self.model.config.id2label)
22
23     def predict(self, claim: str, evidence: str) -> dict:
24         return self.predict_batch(claim, [evidence])[0]
25
26     def predict_batch(self, claim: str, evidence_texts: list[str]) -> list[dict]:
27         claim = claim.strip()
28         if not claim:
29             raise ValueError("Claim must not be empty.")
30
31         evidence_texts = [text.strip() for text in evidence_texts]
32         if any(not text for text in evidence_texts):
33             raise ValueError("Evidence text must not be empty.")
34         if not evidence_texts:
35             return []
36
37         inputs = self.tokenizer(
38             evidence_texts,
39             [claim] * len(evidence_texts),
40             return_tensors="pt",
41             truncation=True,
42             max_length=512,
43             padding=True,
44         )
45
46         with torch.no_grad():
47             outputs = self.model(**inputs)
48
49         probabilities = torch.softmax(outputs.logits, dim=-1)
50         predicted_ids = torch.argmax(probabilities, dim=-1)
51
52         results = []
53         for row_probabilities, predicted_id_tensor in zip(probabilities, predicted_ids):
54             predicted_id = predicted_id_tensor.item()
55             results.append(
56                 {
57                     "label": self.id2label[predicted_id],
58                     "confidence": row_probabilities[predicted_id].item(),
59                     "probabilities": row_probabilities.tolist(),
60                 }
61             )
62         return results
63
64     def convert_to_fever_label(self, nli_label: str) -> str:
65         label = nli_label.lower()
66
67         if "entail" in label:
68             return "SUPPORTS"
69         if "contradiction" in label or "contradict" in label:
70             return "REFUTES"
71         return "NOT ENOUGH INFO"
72
73     def analyze_passage(self, claim: str, passage: dict) -> dict:
74         return self.analyze_passages(claim=claim, passages=[passage])[0]
75
```

Vérification NLI

Le Verifier (verification/verifier.py) utilise un modèle NLI (nli-deberta-v3-base) pour classer chaque paire (preuve, affirmation) en entailment (confirme), contradiction (contredit) ou neutral (ne permet pas de statuer). Ces labels sont convertis vers la nomenclature FEVER (SUPPORTS, REFUTES, NOT ENOUGH INFO). Les prédictions obtenues pour l'ensemble des preuves retenues sont ensuite agrégées (somme des scores de confiance par label) pour déterminer le verdict final et son niveau de confiance.

7.7 Génération de l'explication

```
1
2 def initialize_state() -> None:
3     st.session_state.setdefault("page", PAGE_CHECK)
4     st.session_state.setdefault("claim_text", "")
5     st.session_state.setdefault("current_result", None)
6     st.session_state.setdefault("history", [])
7
8
9 def set_page(page: str) -> None:
10     st.session_state.page = page
11
12
13 def set_claim(claim: str) -> None:
14     st.session_state.claim_text = claim
15
16
17 def random_claim() -> None:
18     st.session_state.claim_text = random.choice(EXAMPLE_CLAIMS)
19
20
21 def clear_claim() -> None:
22     st.session_state.claim_text = ""
23
24
25 def label_class(label: str) -> str:
26     if label == "SUPPORTS":
27         return "supports"
28     if label == "REFUTES":
29         return "refutes"
30     return "nei"
31
32
33 def label_display(label: str) -> str:
34     if label == "SUPPORTS":
35         return "CONFIRME"
36     if label == "REFUTES":
37         return "CONTREDIT"
38     return "PREUVES INSUFFISANTES"
39
40
41 def percent(value: float | None) -> str:
42     return f"({value or 0.0} * 100:.0f)%"
43
44
45 def label_progress_class(label: str) -> str:
46     return label_class(label)
47
48
49 def truncate_text(text: str, length: int = 130) -> str:
50     text = " ".join(str(text).split())
51     if len(text) <= length:
52         return text
53     return f"{text[: length - 1].rstrip()}..."
54
55
56 @st.cache_data(show_spinner=False)
57 def load_artifact_status() -> dict:
58     status = {
59         "faiss_index": FAISS_INDEX_FILE.exists(),
60         "sqlite_database": CHUNKS_DATABASE_FILE.exists(),
61         "processed_chunks": PROCESSED_CHUNKS_FILE.exists(),
62         "embeddings_metadata": EMBEDDINGS_METADATA_FILE.exists(),
63         "chunk_count": None,
64         "document_count": None,
65     }
66
67     if CHUNKS_DATABASE_FILE.exists():
68         connection = sqlite3.connect(CHUNKS_DATABASE_FILE)
69         try:
70             row = connection.execute(
71                 "SELECT COUNT(*), COUNT(DISTINCT doc_id) FROM chunks"
72             ).fetchone()
73             status["chunk_count"] = row[0]
74             status["document_count"] = row[1]
75         finally:
76             connection.close()
77
78     return status
79
80
81 @st.cache_resource(show_spinner="Chargement des modèles de recherche, de tri et d'analyse...")
82 def load_model_bundle() -> tuple[DocumentRetriever, PassageRetriever, Reranker, Verifier]:
83     document_retriever = DocumentRetriever()
84     passage_retriever = PassageRetriever(document_retriever)
85     reranker = Reranker()
86     verifier = Verifier()
87     return document_retriever, passage_retriever, reranker, verifier
88
89
90 def load_pipeline(
91     retrieval_top_k: int,
92     rerank_top_k: int,
93     max_chunks_per_document: int,
94 ) -> FactCheckerPipeline:
95     document_retriever, passage_retriever, reranker, verifier = load_model_bundle()
96     settings = PipelineSettings(
97         retrieval_top_k=retrieval_top_k,
98         rerank_top_k=rerank_top_k,
99         max_chunks_per_document=max_chunks_per_document,
100     )
101     return FactCheckerPipeline(
102         document_retriever=document_retriever,
103         passage_retriever=passage_retriever,
104         reranker=reranker,
105         verifier=verifier,
106         settings=settings,
107     )
108
```

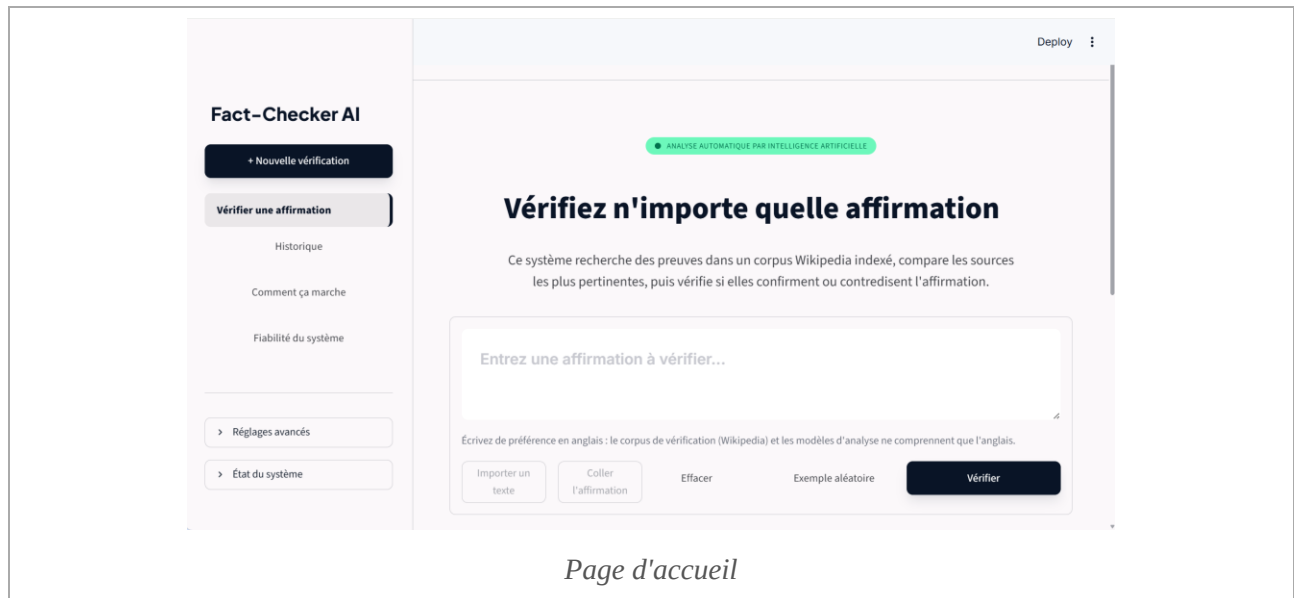
Explication du verdict

L'interface Streamlit (frontend/app.py) restitue, pour chaque verdict, le document source, le passage utilisé et le niveau de confiance de l'analyse pour chaque preuve retenue. L'« explication » fournie par le système repose ainsi sur la transparence des preuves et des scores associés, plutôt que sur un texte explicatif généré par un modèle de langage génératif.

8. Interface utilisateur

L'interface, développée avec Streamlit (frontend/app.py), est organisée en plusieurs pages accessibles depuis un menu latéral : « Vérifier une affirmation », « Résultat », « Historique », « Comment ça marche » et « Fiabilité du système ». Un panneau de réglages avancés permet d'ajuster le nombre de sources explorées, le nombre de preuves retenues et le nombre maximal de passages par document source.

Page d'accueil / Vérifier une affirmation



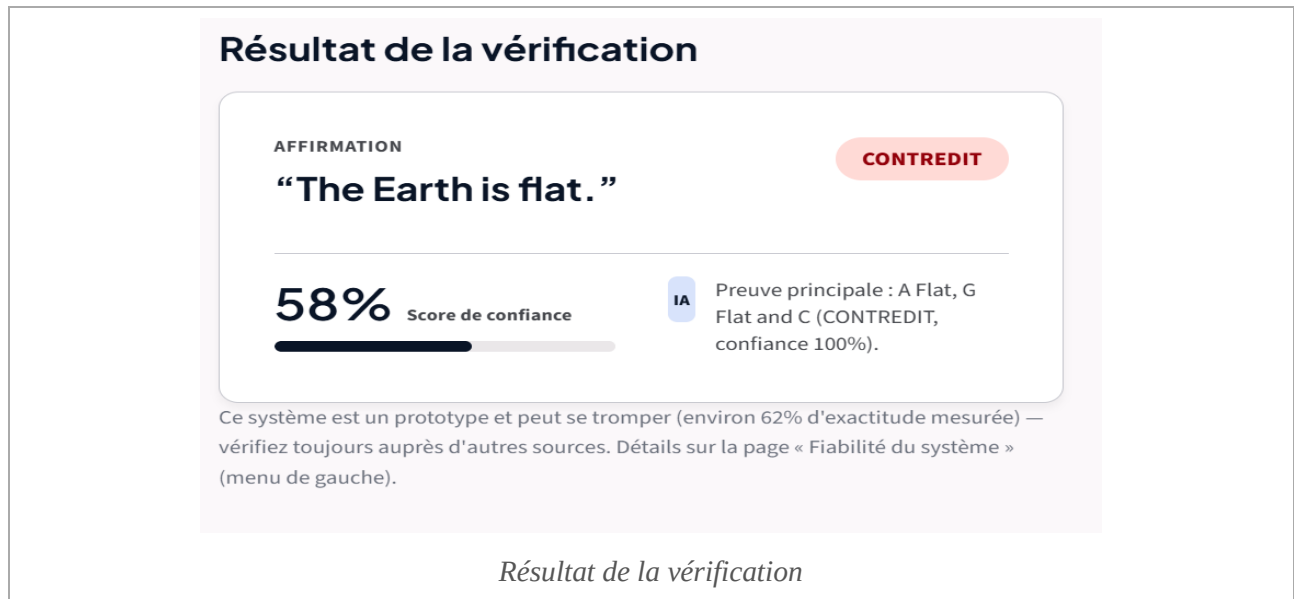
Cette page présente le principe du système et permet de saisir une affirmation à vérifier.

Saisie d'une affirmation



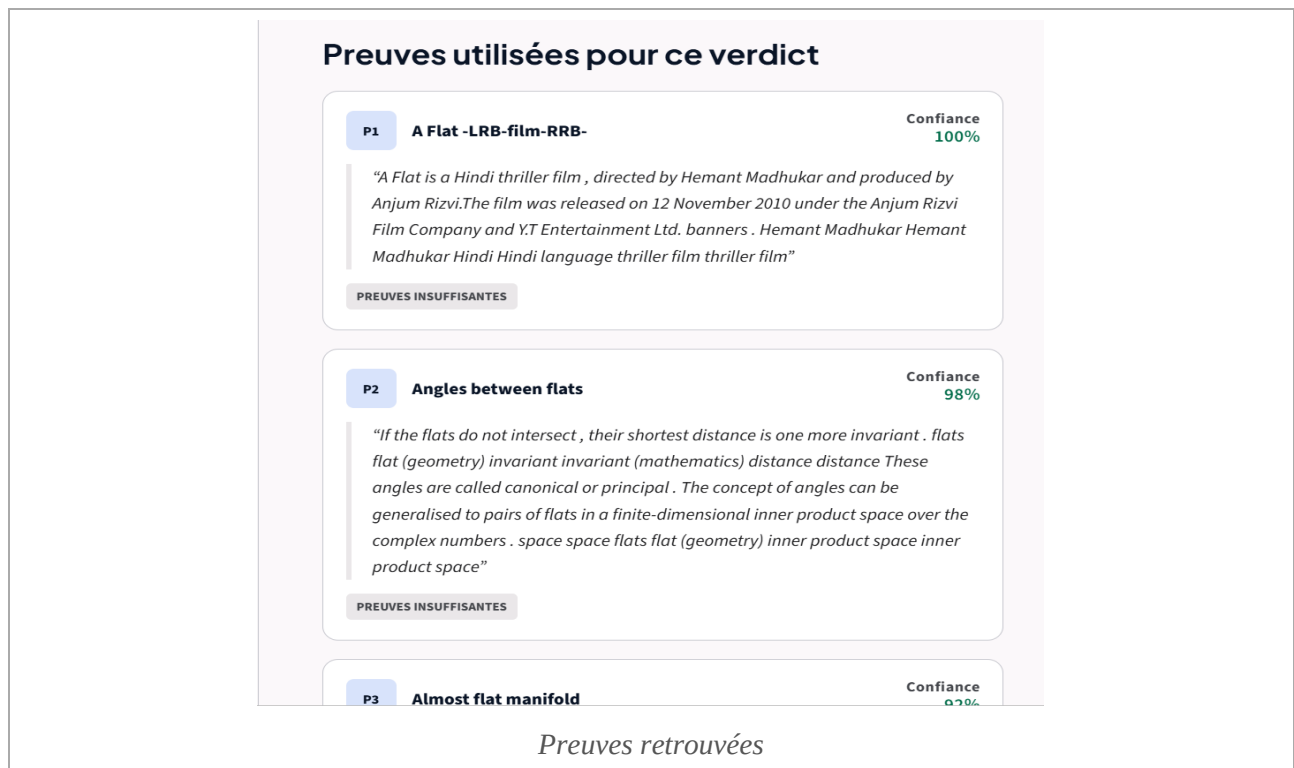
Un champ de texte permet de saisir librement une affirmation, avec des exemples cliquables proposés par défaut. L'interface précise explicitement que le corpus et les modèles ne comprennent que l'anglais.

Résultat



Le verdict final (Confirme / Contredit / Preuves insuffisantes) est affiché avec son score de confiance sous forme de progression.

Preuves



Chaque preuve retenue est présentée sous forme de carte indiquant le document source, le passage extrait et le score de confiance associé à son analyse individuelle.

Explication / Comment ça marche



Cette page décrit visuellement les six étapes du pipeline (Affirmation, Recherche, Tri, Preuves, Analyse, Verdict) ainsi que la taille de la base de connaissances disponible.

9. Tests et résultats

Exemples de vérification

Les exemples suivants ont été obtenus en exécutant le pipeline complet (FactCheckerPipeline.verify_claim) sur l'environnement de développement (CPU, sans GPU) :

Affirmation	Verdict	Confiance	Temps d'exécution
The Eiffel Tower is located in Paris.	SUPPORTS	80 %	79,96 s
Albert Einstein won the Nobel Prize in Physics.	SUPPORTS	60 %	32,66 s
The Earth is flat.	REFUTES	58 %	22,66 s

Temps moyen observé par affirmation : 45,09 secondes (hors chargement initial des modèles). Le chargement initial des trois modèles (embeddings, reranker, NLI) prend environ 66,73 secondes.

Évaluation sur le jeu de données FEVER

Le pipeline complet a été évalué sur le jeu de développement FEVER (shared_task_dev.jsonl), à l'aide du script evaluation/evaluate_pipeline.py. La dernière exécution a chargé 19 998 exemples et détecté 96 406 documents distincts dans la base SQLite des chunks. Avec une limite fixée à 50 affirmations évaluables, seuls 24 exemples ont effectivement pu être évalués, tandis que 19 974 ont été ignorés, principalement parce que leurs preuves de référence étaient absentes du corpus indexé courant.

Métrique	Valeur	Signification
Accuracy	75,00 %	Part des verdicts finaux corrects
Evidence Recall	54,17 %	Part des affirmations pour lesquelles une preuve correcte a été retrouvée
Evidence Precision	3,34 %	Part des passages retenus qui correspondent exactement à une preuve de référence
FEVER Score	41,67 %	Verdict ET preuve corrects simultanément (métrique officielle FEVER)
Macro F1	0,5211	Moyenne des F1 par classe (SUPPORTS / REFUTES / NOT ENOUGH INFO)
ECE (Expected Calibration Error)	0,1333	Écart entre la confiance annoncée et la fiabilité réelle du modèle NLI

Exemples d'erreurs observées

L'analyse des affirmations mal classées montre plusieurs cas où le modèle NLI est très confiant (proche de 100 %) sur un label pourtant incorrect, révélant un problème de sur-confiance plutôt qu'un simple manque de preuves :

Affirmation	Label réel	Label prédit	Confiance
Damon Albarn has released something.	SUPPORTS	REFUTES	99,44 %
Aleister Crowley was European.	SUPPORTS	NOT ENOUGH INFO	99,93 %
Chris Mullin played with a team based in the United States.	SUPPORTS	NOT ENOUGH INFO	99,94 %
The Africa Cup of Nations is a friendly global soccer exhibition.	REFUTES	NOT ENOUGH INFO	98,53 %
John Goodman starred in 10 Cloverfield Lane.	SUPPORTS	REFUTES	99,94 %
Jimi Hendrix received training for air assault operations.	SUPPORTS	NOT ENOUGH INFO	99,50 %

10. Difficultés rencontrées

- Volume important des données Wikipedia à traiter, avec un risque de saturation mémoire lors du prétraitement.
- Génération des embeddings sur un grand nombre de passages, nécessitant une gestion efficace des lots et du stockage.
- Gestion de l'environnement et des dépendances d'apprentissage automatique (PyTorch, Transformers, Sentence-Transformers, FAISS) : leur installation directe sur l'environnement Windows du poste de développement s'est révélée impossible, ce qui a nécessité l'utilisation d'un environnement conda dédié sous WSL (Windows Subsystem for Linux).
- Temps de calcul important en l'absence de GPU : le chargement des modèles (environ 67 secondes) et le traitement d'une affirmation (20 à 80 secondes observées) restent lents pour un usage interactif.

11. Solutions apportées

- Lecture en streaming des fichiers Wikipedia et génération progressive des chunks (article par article), afin de limiter l'empreinte mémoire du prétraitement.
- Encodage des embeddings par lots (`BATCH_SIZE = 512`) et sauvegarde fragmentée en plusieurs fichiers `.npy` accompagnés d'un fichier de métadonnées, plutôt qu'un unique fichier volumineux.
- Mise en place d'un environnement conda dédié (« fact-checker ») sous WSL, isolant les dépendances d'apprentissage automatique de l'environnement Windows natif.
- Ajout de nouvelles métriques d'évaluation (Macro F1 et Evidence Precision) dans `evaluation/metrics.py` et leur intégration dans `evaluation/evaluate_pipeline.py`, afin de compléter l'analyse de performance du système.
- Utilisation d'un index FAISS de type IndexFlatIP combiné à des embeddings normalisés, permettant une recherche par similarité cosinus efficace pour l'étape de recherche, indépendamment du coût plus élevé de l'étape de vérification NLI.

12. Limites

- Dépendance à Wikipedia : le système ne peut vérifier que des affirmations dont les preuves sont présentes dans le corpus indexé ; il ne consulte aucune source externe ni le Web en temps réel.
- Couverture limitée du corpus : dans la configuration actuelle du prétraitement (`MAX_FILES = 2` dans `indexing/preprocess_wikipedia.py`), seul un sous-ensemble des fichiers Wikipedia est traité. La dernière évaluation a détecté 96 406 documents distincts dans la base des chunks. De nombreuses affirmations du jeu FEVER ont dû être ignorées lors de l'évaluation car leurs preuves de référence ne figuraient pas dans le corpus indexé.

- Performances variables selon les affirmations : sur l'échantillon évalué, le système atteint 75,00 % d'exactitude et 41,67 % de FEVER Score, avec des cas de sur-confiance du modèle NLI sur des prédictions erronées.
- Faible précision des preuves (Evidence Precision de 3,34 % sur l'échantillon testé) : parmi les passages retenus après reranking, seule une faible proportion correspond exactement à une phrase de preuve de référence.
- Temps de calcul : en l'absence de GPU, le traitement d'une affirmation prend de 20 à 80 secondes environ (hors chargement des modèles), ce qui limite l'usage en temps réel à grande échelle.
- Langue unique : le corpus documentaire et les modèles utilisés ne traitent que l'anglais, comme indiqué explicitement dans l'interface utilisateur.

13. Perspectives

- Support de plusieurs langues, au-delà de l'anglais actuellement requis.
- Recherche Web en temps réel, en complément du corpus Wikipedia statique actuellement indexé.
- Utilisation de modèles de langage plus récents, notamment pour améliorer la robustesse du module de vérification NLI et réduire les cas de sur-confiance observés.
- Amélioration du pipeline RAG (recherche augmentée), par exemple en enrichissant la stratégie de reranking ou en indexant l'intégralité du corpus Wikipedia plutôt qu'un sous-ensemble.
- Déploiement dans le cloud, pour un accès mutualisé et une meilleure disponibilité.
- Exposition du pipeline via une API (par exemple avec FastAPI), pour permettre une intégration dans d'autres applications au-delà de l'interface Streamlit actuelle.

Conclusion

Ce projet a permis de concevoir et de mettre en œuvre un système complet de vérification automatique de faits, depuis le prétraitement du corpus Wikipedia jusqu'à la restitution d'un verdict argumenté via une interface utilisateur. L'ensemble des objectifs spécifiques fixés a été atteint : prétraitement du corpus, construction de la base documentaire et de l'index FAISS, recherche et tri des preuves pertinentes, vérification par un modèle NLI, et présentation des résultats sous forme de verdict, de preuves et de score de confiance.

L'évaluation menée sur le jeu de développement FEVER montre des résultats encourageants pour un prototype sur les exemples couverts par l'index courant : 24 affirmations évaluées, 75,00 % d'exactitude, 54,17 % de rappel des preuves, 41,67 % de FEVER Score et 0,5211 de Macro F1. Elle révèle toutefois des marges de progression importantes, notamment sur la précision des preuves retrouvées (3,34 %) et sur la confiance parfois mal calibrée du module de vérification (ECE de 0,1333).

Les limites identifiées — couverture partielle du corpus, dépendance à Wikipedia, temps de calcul sur CPU — ouvrent des perspectives claires d'amélioration : élargissement du corpus indexé, intégration de sources complémentaires, adoption de modèles plus récents, et déploiement à plus grande échelle. Ce travail constitue ainsi une base fonctionnelle pour un système de fact-checking assisté par IA, tout en illustrant concrètement les défis techniques propres aux architectures de type recherche augmentée (RAG).